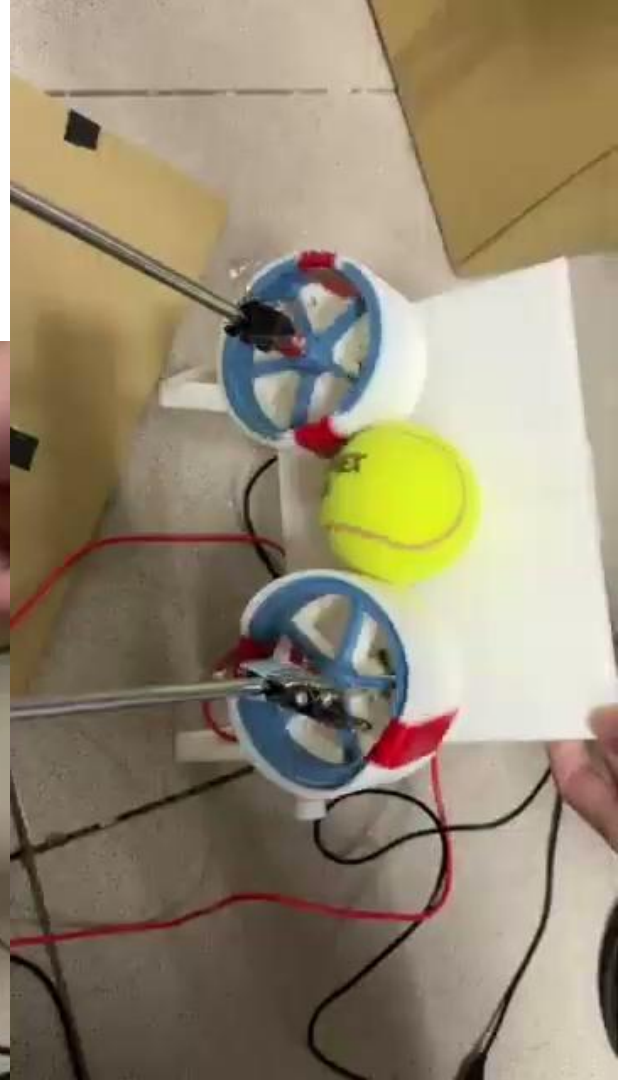
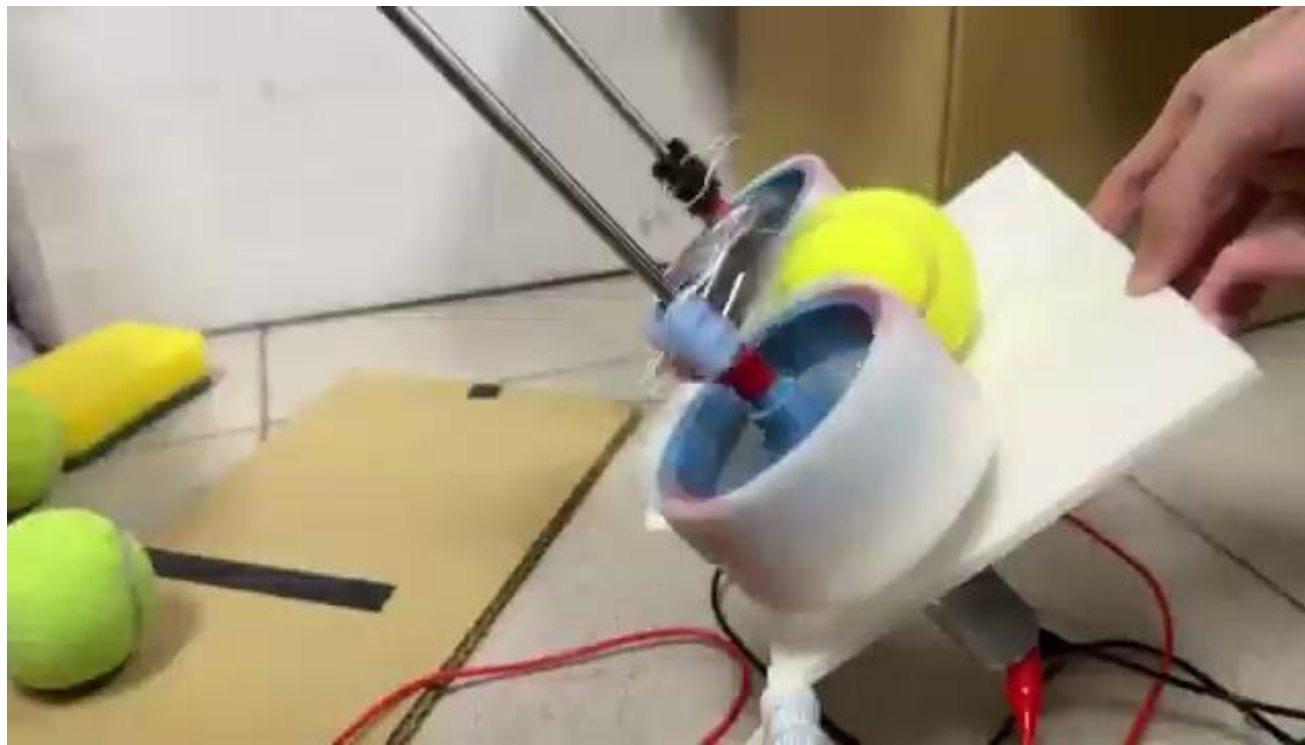
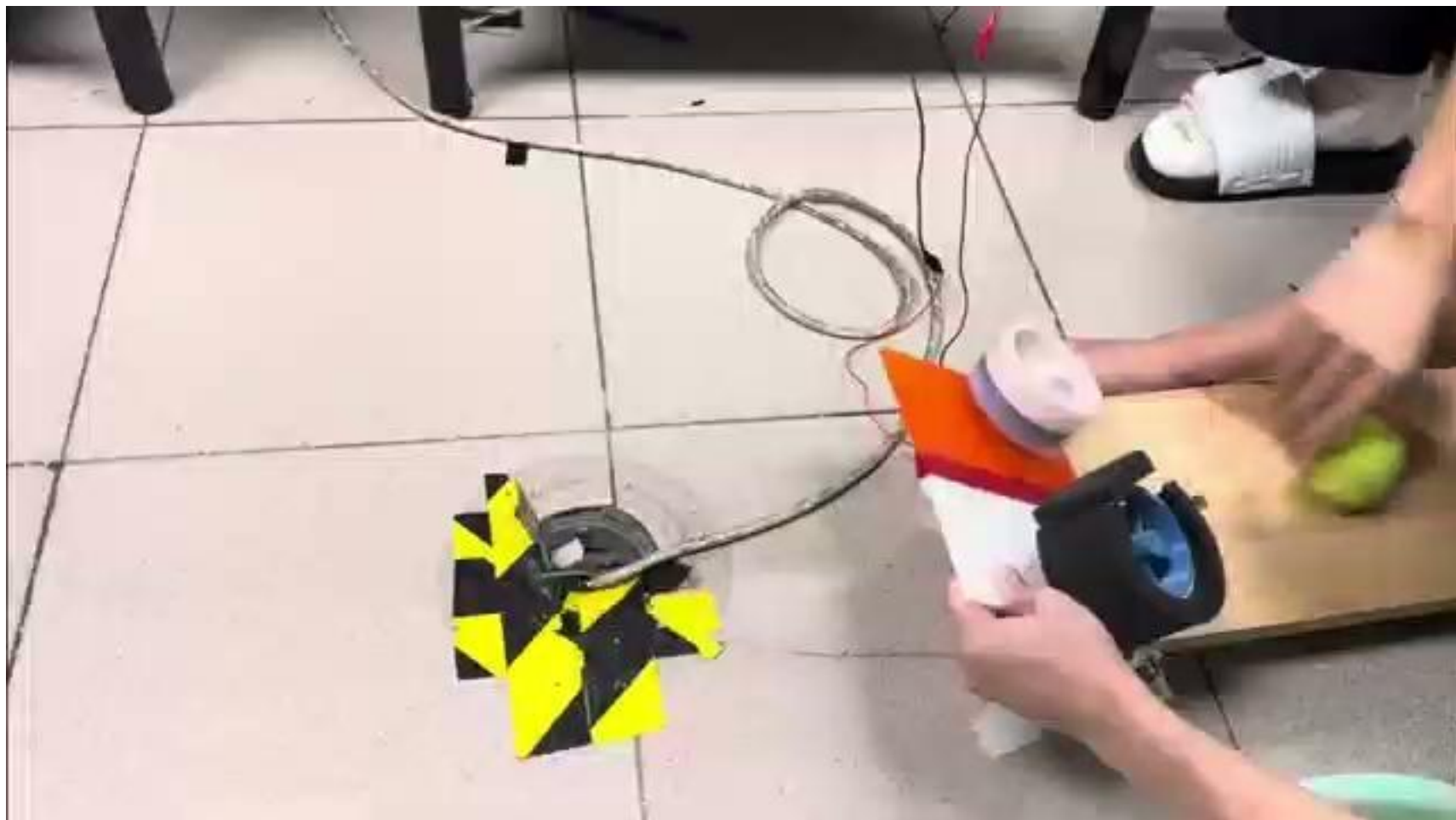


撿球機

3/26

用另一顆球頂開





改善版撿球實驗

使用24v、5200rpm的馬達(實際供電為18v)
轉動單邊輪胎，另一邊不用馬達

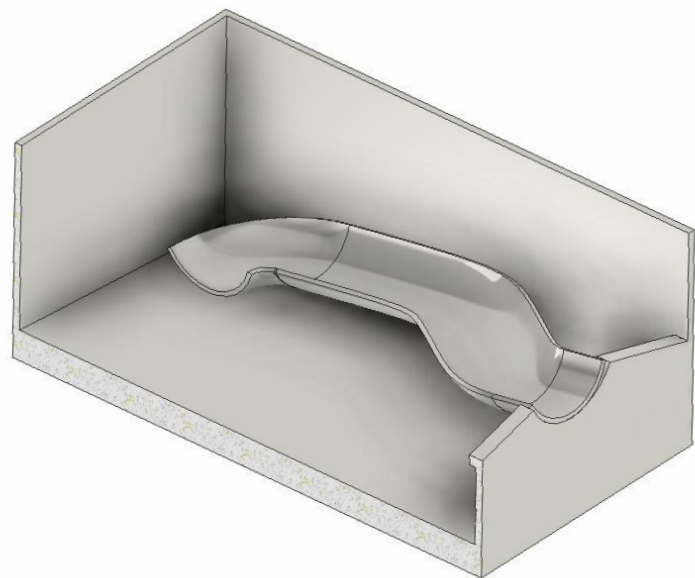
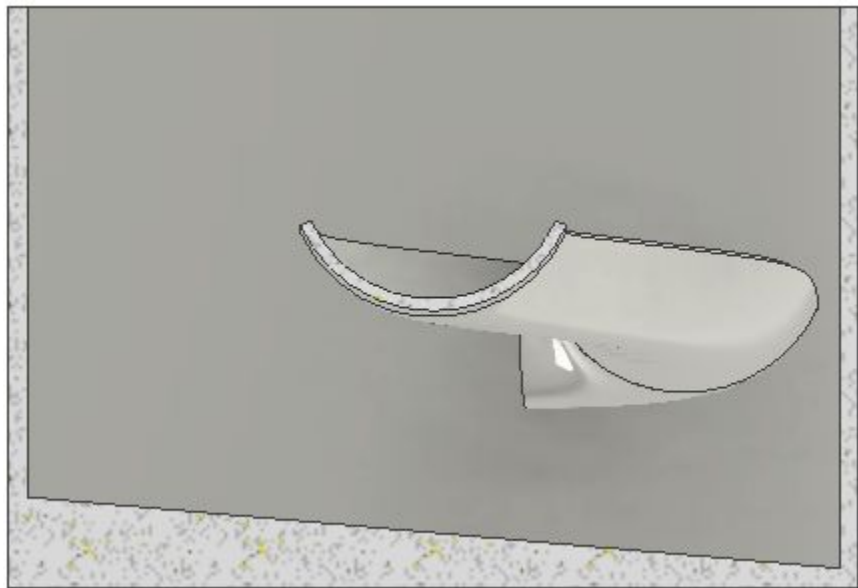
球可以射出超過理想距離

但馬達扭力過大會使沒轉動的滾輪連接軸斷裂



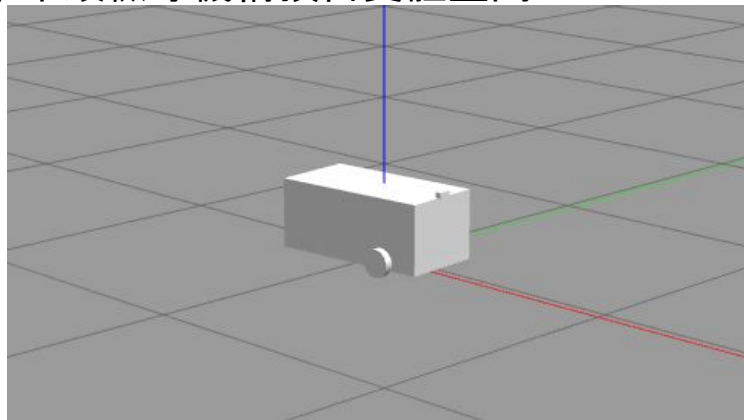
下次目標

1. 加固滾輪連接軸
2. 測試供電幾伏可以讓射出距離在理想距離內
3. 滾輪表面材質選擇, 改用較厚的海綿或是直接不貼其他材質

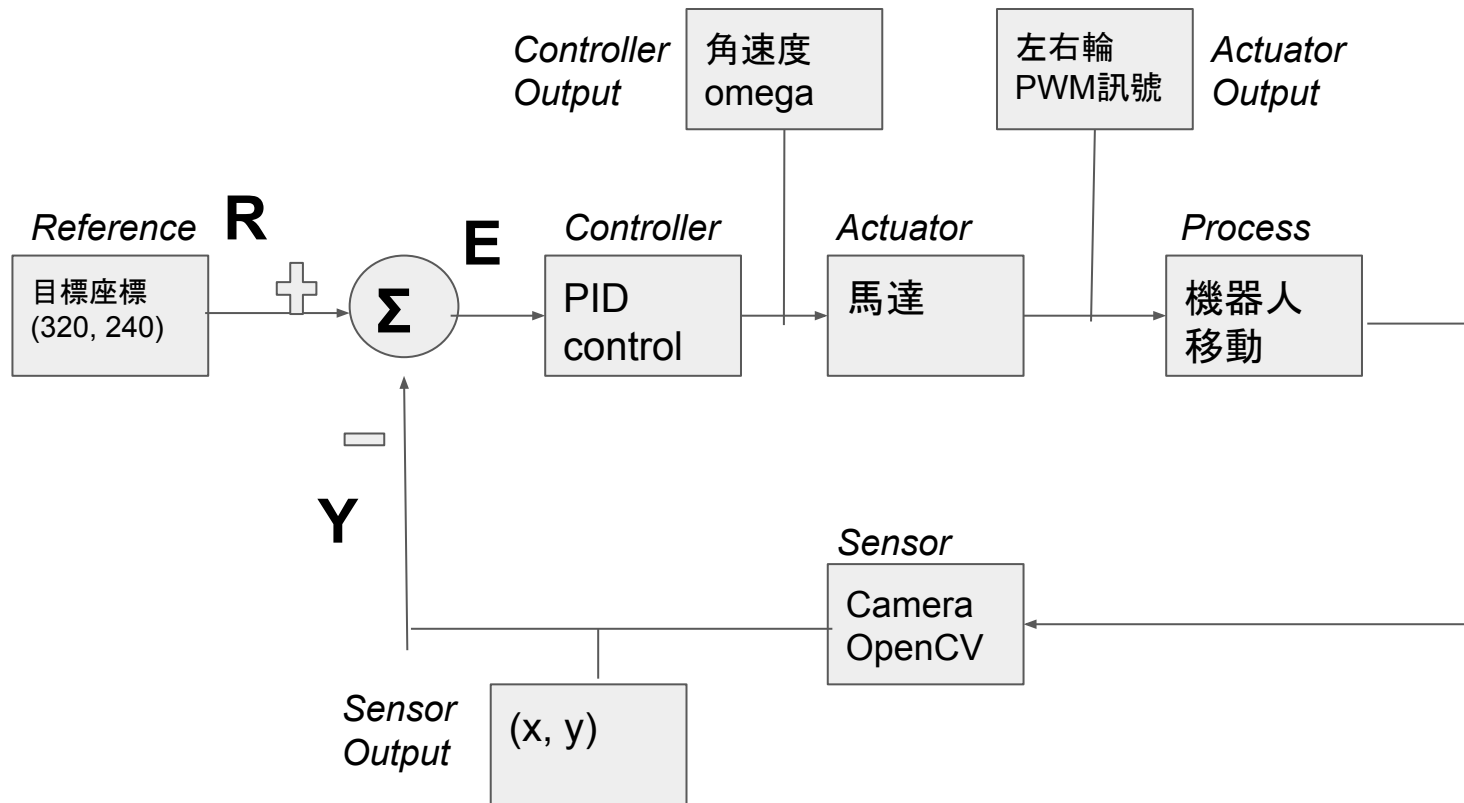


數位孿生(Digital Twin)軟體架構建置

1. 建構 ROS 2 + Gazebo數位孿生軟體架構
2. 完成視覺伺服閉迴路(尋球、鎖定、移動、煞車)
3. 先採用 OpenCV 色域過濾, 推遲 YOLO 整合
4. 兩輪差速驅動 + 車尾萬向輔助輪
5. 車體尺寸: 600 x 300 x 250 mm
6. 驅動輪: 直徑 100 mm
7. 離地間隙: 25 mm
8. 機構防呆: 底盤 < 網球直徑 (66 mm) -> Error=0, 為未來車頭檢球機構預留實體空間
9. 感測器: 640 x 480 解析度, 30 FPS 虛擬攝影機



方向_控制架構圖



P-Controller 與像素誤差計算

目標 Reference (R): 攝影機畫面正中央(解析度 640*480, 目標 X=320)。

回授 Feedback (Y): OpenCV 算出網球中心點 (x,y)。

誤差 Error (E): $E = R - Y = (320 - x, 240 - y)$

控制器 Output (U):

- **轉向角速度 (Ω):** $\Omega = E * K_p$ (目前 $K_p = 0.005$)
- **線速度 (v):** 根據網球黃色區塊面積 (Area) 設定閾值。當 $Area > 30000$ 時, 判定夠近, 線速度歸零煞車。

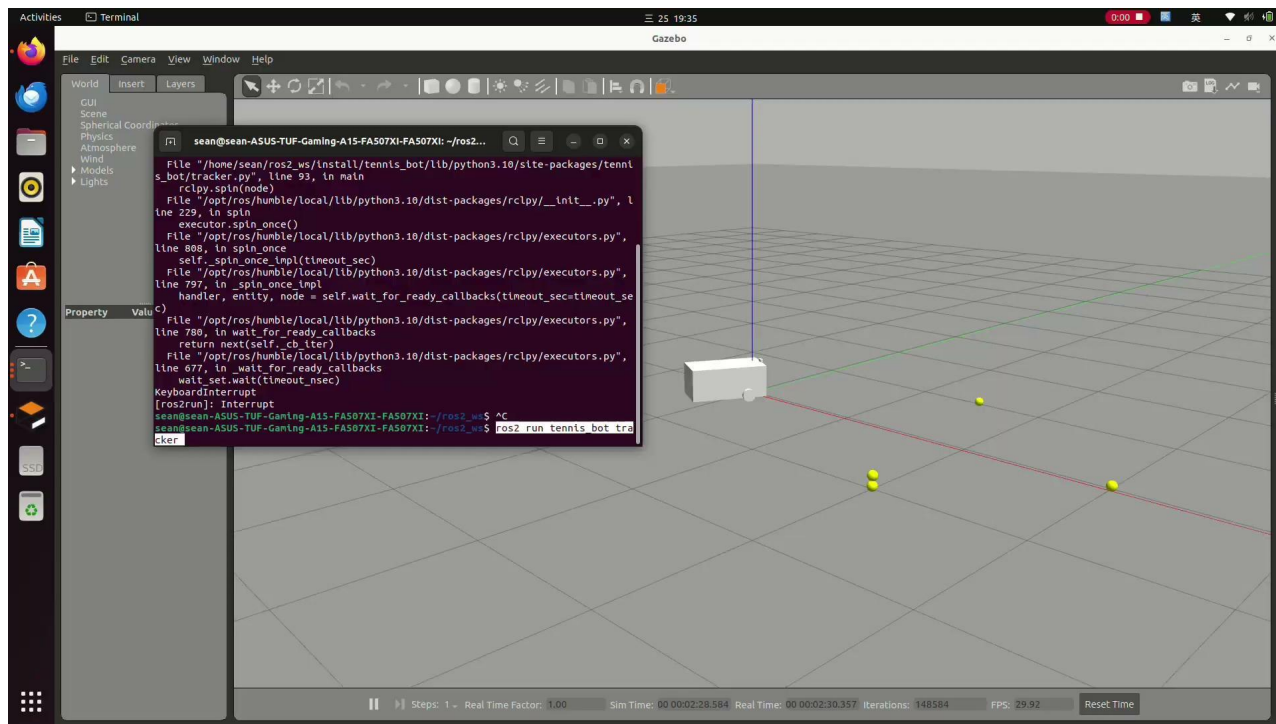
閉迴路實作&下次目標

自動尋標->像素誤差計算->轉向鎖定->接近煞車

下次目標

Pcontrol -> PID

OpenCV -> YOLO



Random Split → K-Fold

- 原本方法: 70% / 30% Random Split
 - `random.shuffle(all_images)`
 - 圖片切成 70% train / 30% validation
 - 重複多次實驗
 - 同一張圖片可能: 在某一輪是 train、在另一輪變成 validation
- 改進方法: K-Fold Cross-Validation (K=5)
 - 將資料平均分成 5 份 (fold_1 ~ fold_5)
 - 每一輪: 1 份當 validation、其餘 4 份當 train
 - 共訓練 5 次, 最後取平均結果

資料重複使用且分布不固定

資料都當過 validation → 公平
評估結果更穩定 → variance
符合「交叉驗證」

提升推論速度

方法一：模型結構優化（未採用）

- 修改 convolution / backbone → 使用 YOLO 官方模型（已高度優化）
→ 需修改架構需重新設計與訓練

方法二：輸入與模型配置調整

- 輸入尺寸 (imgsz)
 - imgsz ↑ → mAP ↑ (細節更多); imgsz ↑ → FPS ↓ (計算量大)
- 模型格式 (.pt vs .onnx)
 - 可搭配加速後端 (TensorRT / CUDA)
 - 理論上 **推論速度** ↑

調整輸入影像尺寸

- 訓練設定

- 使用 3 種影像尺寸訓練模型：
- 416 / 512 / 640 pixel

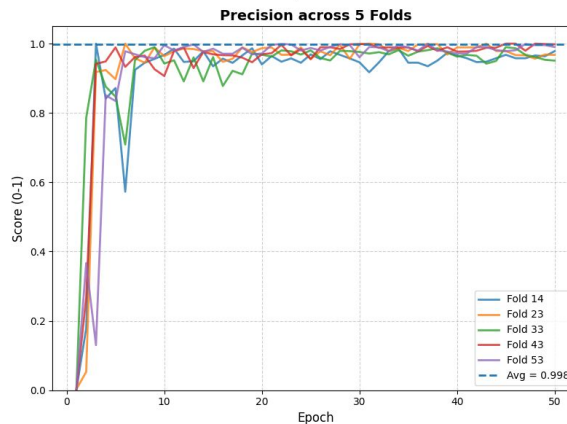
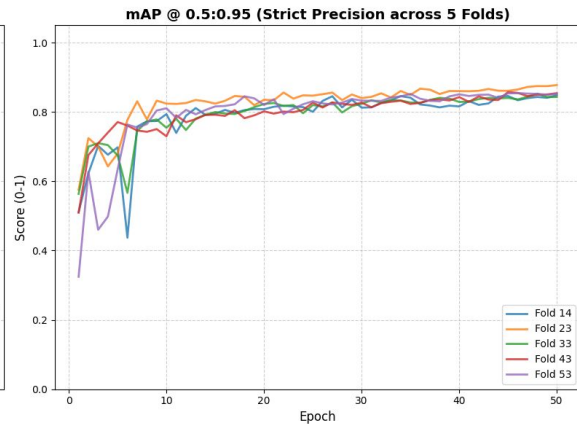
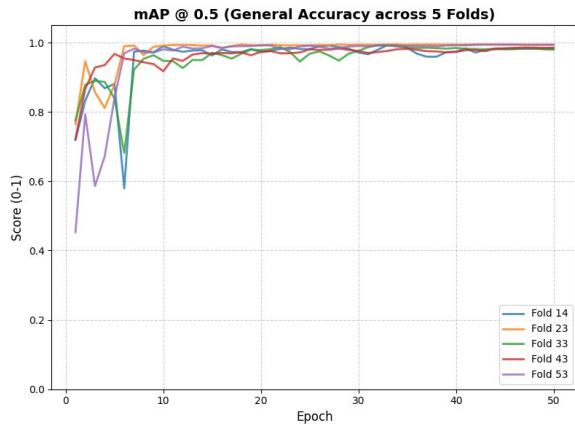
- 測試設定

- 每個模型搭配 3 種輸入尺寸：
- 416 / 512 / 640

→ 共 $3 \times 3 = 9$ 種組合

- 評估

- 速度：FPS
- 準確度 (mAP50、mAP50-95、Precision、Recall)



訓練圖片416pixel模型準確度參考圖

調整輸入影像尺寸(imgsz)

	FPS (Test Size:640)	FPS (Test Size:512)	FPS (Test Size:416)	mAP50	mAP50-95	Precision	Recall
Train size:640	8.016	10.686	13.486	0.9897	0.8591	0.9982	0.981
Train size:512	7.846	10.204	13.416	0.9911	0.8535	0.9972	0.986
Train size:416	8	10.472	13.614	0.992	0.858	0.998	0.9883

- 整體準確度差異非常小
- 測試尺寸越小 → FPS 越高

約略提升幅度：

- 640 → 416:約 65% ~ 70%
- 512 → 416:約 25% ~ 30%

	.pt			onnx			
	FPS (Test Size:640)	FPS (Test Size:512)	FPS (Test Size:416)	FPS (Test Size:640)	FPS (Test Size:512)	FPS (Test Size:416)	提升幅度
Train size:640	8.016	10.686	13.486	7.168	9.63	16.026	+19%
Train size:512	7.846	10.204	13.416	6.774	10.106	14.036	+4%
Train size:416	8	10.472	13.614	6.876	9.582	15.814	+16%

在 Test Size = 416 .onnx 明顯更快

ONNX可接加速後端 →效果可能更好

未來展望：

模型由 **FP32** → **FP16** → **INT8**